

SISU Technologies | Prueba en casa Desarrollo Full-Stack

1. Objetivo

Construir una aplicación web sencilla con temática de apuestas en carreras de caracoles.

La prueba busca conocer cómo organizas una solución, cómo te adaptas a un stack determinado, qué prácticas de desarrollo aplicas, cómo manejas diferentes respuestas de una integración y qué tan claramente puedes explicar tus decisiones.

La prueba deberá desarrollarse utilizando:

- React para el frontend.
- Express para el backend.
- TypeScript tanto en frontend como en backend.
- LocalStorage para conservar la información del usuario, la sesión y el saldo.

Solicitamos específicamente este stack para evaluar tu facilidad de adaptación y respuesta ante un contexto tecnológico determinado.

Tendrás siete días calendario para realizar la entrega. Esperamos que el alcance principal pueda resolverse en no más de seis a ocho horas de trabajo. No buscamos una aplicación lista para producción ni esperamos que dediques toda la semana a desarrollarla.

Puedes entregar la prueba aunque no hayas completado el 100 % de las funcionalidades. En ese caso, deberás indicar claramente qué terminaste, qué quedó pendiente y cómo continuarías el desarrollo, cualquier discrepancia entre el código y las funcionalidades declaradas como finalizadas será penalizada.

2. Funcionalidades

2.1 Registro e inicio de sesión

La aplicación deberá permitir registrar un usuario utilizando:

- Nombre completo.
- Correo electrónico.
- Contraseña.
- Confirmación de contraseña.

El formulario deberá contar con las validaciones que consideres necesarias y no deberá solicitar ni procesar archivos adjuntos.

Una vez registrado, el usuario deberá poder:

- Acceder a la aplicación.
- Cerrar su sesión.
- Iniciar sesión nuevamente utilizando su correo y contraseña.
- Mantener su información después de recargar la página.

- Acceder al dashboard únicamente cuando exista una sesión activa.

No es necesario implementar:

- Recuperación de contraseña.
- Verificación de correo electrónico.
- Administración de múltiples usuarios.

Para esta prueba, el registro y el inicio de sesión serán una simulación local. La forma en que decidas tratar y almacenar la contraseña será parte de la evaluación.

El usuario deberá iniciar con un saldo de \$0.

2.2 Dashboard

Después de iniciar sesión, deberá mostrarse un dashboard. La organización, apariencia y distribución quedan a tu elección, pero deberá contener como mínimo:

- Nombre del usuario registrado.
- Saldo actual.
- Una gráfica tipo donut de apuestas ganadas y perdidas.
- Una gráfica de barras con las victorias de los caracoles.
- Una opción para cargar saldo mediante la pasarela simulada SnailPay.
- Una opción para cerrar sesión.

Los datos de apuestas y carreras deben ser simulados. No deberás construir una sección para realizar apuestas ni una lógica para ejecutar carreras.

Gráfica de apuestas

La gráfica tipo donut deberá simular la cantidad de apuestas ganadas y perdidas.

Gráfica de carreras

La gráfica de barras deberá representar las victorias obtenidas durante un día simulado:

- Deberán existir 6 caracoles.
- Puedes elegir libremente sus nombres.
- Se realizan seis carreras durante el día.
- Los datos mostrados deben ser simulados, pero deben tener congruencia con las reglas y parámetros.

Puedes utilizar la librería de gráficas que prefieras.

2.3 Carga de saldo con SnailPay

Deberás construir en Express un servicio simulado llamado SnailPay.

SnailPay es un mock de una pasarela de pagos. No deberá conectarse con servicios reales ni procesar

información financiera real.

La integración deberá solicitar mediante un API:

- Número de tarjeta.
- Fecha de vencimiento.
- CVV.
- Nombre completo.
- Monto de la recarga.

También deberá utilizar el identificador y el correo del usuario registrado.

El servicio deberá contemplar los siguientes resultados:

2.3.1. Cobro exitoso

Los siguientes datos deberán producir un cobro exitoso:

- Número de tarjeta: 1234123412341234
- Fecha de vencimiento: 12/26
- CVV: 543
- Nombre completo: cualquier valor no vacío.
- Monto: cualquier cantidad válida mayor que cero.

Cuando el cobro sea exitoso:

- El saldo deberá aumentar por el monto solicitado.
- El nuevo saldo deberá guardarse en LocalStorage.
- El dashboard deberá mostrar inmediatamente el saldo actualizado.
- Deberá informarse claramente al usuario que la operación fue aprobada.

2.3.2. Error en la transacción

Deberás decidir qué errores de transacción deseas simular, qué datos los provocan y qué información resulta útil devolver en `status_detail`.

Puedes incluir uno o más escenarios, por ejemplo: datos inválidos, tarjeta rechazada u otra situación que consideres relevante.

2.3.3. Error del sistema

Deberá existir una forma documentada de simular que SnailPay tiene un problema interno y no puede procesar solicitudes.

Durante esta simulación no deberá aprobarse ni aplicarse ninguna recarga.

2.4. Respuestas de SnailPay

Las respuestas de cobro exitoso, error de transacción y error del sistema deberán contener los siguientes

campos:

Campo	Descripción
<code>id</code>	Identificador de la operación
<code>status</code>	Estado general de la operación
<code>status_detail</code>	Detalle del resultado
<code>transaction_amount</code>	Monto solicitado
<code>date_created</code>	Fecha de creación de la operación
<code>authorization_code</code>	Código de autorización cuando corresponda
<code>reference</code>	Referencia de la operación
<code>payer_id</code>	Identificador del usuario
<code>payer_email</code>	Correo del usuario

Puedes definir los valores y formatos específicos, siempre que sean consistentes y estén documentados.

Cuando una operación no sea exitosa:

- El saldo no deberá modificarse.
- El usuario deberá recibir un mensaje comprensible.
- No deberán generarse falsos cobros exitosos.

El número de tarjeta y el CVV:

- Deberán incluirse en las respuestas del servicio.
- Deberán guardarse en `localStorage`.
- Siempre deberán ser datos ficticios.

3. Diseño y herramientas permitidas

Puedes utilizar una plantilla, librería de componentes, sistema visual o herramienta de generación de interfaces.

Deberás indicar en tu documento de respuesta:

- Qué plantilla o herramienta utilizaste.
- Qué partes fueron generadas o tomadas como base.
- Qué partes adaptaste o construiste personalmente.

No buscamos un diseño de alta fidelidad, pero sí una interfaz clara, consistente y utilizable. Entregar una

aplicación funcional sin trabajo de diseño o presentación disminuirá la calificación correspondiente.

4. Inteligencia artificial

Puedes utilizar herramientas de inteligencia artificial en cualquier parte del proceso, incluyendo:

- Análisis.
- Diseño.
- Generación o modificación de código.
- Pruebas.
- Documentación.
- Investigación.
- Solución de errores.

Utilizar inteligencia artificial no disminuye por sí mismo tu calificación.

Deberás documentar:

- Qué herramientas utilizaste.
- Para qué las utilizaste.
- Qué partes de la solución fueron apoyadas por estas herramientas.
- Cuál fue tu proceso de trabajo.

En las siguientes fases del proceso podremos pedirte explicar, defender o modificar cualquier parte del código entregado. Entregar código que no puedas explicar afectará la evaluación.

5. Buenas prácticas y pruebas

Esperamos que demuestres las buenas prácticas y estándares de desarrollo que actualmente dominas. Esto puede reflejarse en aspectos como:

- Organización del proyecto.
- Separación de responsabilidades.
- Nombres de variables, funciones y componentes.
- Tipado.
- Validaciones.
- Manejo de estados y errores.
- Seguridad.
- Legibilidad.
- Uso de Git.
- Documentación.

Deberás implementar pruebas automatizadas acordes con tu nivel actual de conocimiento.

No solicitamos un porcentaje mínimo de cobertura ni una cantidad determinada de pruebas. Evaluaremos qué decidiste probar, por qué lo consideraste importante y tu capacidad para explicar las pruebas implementadas.

6. Requisitos mínimos de validez

Puedes entregar una solución incompleta. Las funcionalidades terminadas se evaluarán de acuerdo con su nivel de cumplimiento y calidad.

Sin embargo, la entrega será considerada inválida si no contiene como mínimo una aplicación funcional que permita:

1. Registrar un usuario con correo y contraseña.
2. Cerrar la sesión.
3. Iniciar sesión nuevamente con los datos registrados.
4. Acceder a una pantalla posterior al inicio de sesión.

Una entrega que solamente contenga diseños, componentes aislados, código sin ejecutar o documentación no cumplirá con el mínimo requerido.

7. Entregables

Deberás entregar:

1. Un documento de respuesta en formato PDF.
2. Instrucciones suficientes para ejecutar frontend y backend.
3. Instrucciones para ejecutar las pruebas.
4. La información necesaria para reproducir cada respuesta simulada de SnailPay.
5. Una liga a un repositorio público de GitHub con el código.

El repositorio deberá utilizar el siguiente formato de nombre:

`[primer-nombre]-[identificador numérico aleatorio de 4 cifras]`

Ejemplo:

ana-1234

El repositorio y el código no deberán contener nombres, logotipos, enlaces ni referencias que permitan identificar a la empresa o la prueba que realiza este proceso.

Esta medida busca reducir la posibilidad de que otros participantes encuentren las entregas buscando el nombre de la empresa.

Documento PDF de respuesta

El PDF deberá utilizar:

- Fuente Arial de 10 puntos.
- Interlineado estándar.
- Un máximo de cuatro páginas para la entrega principal.

El PDF no deberá contener:

- Código fuente.
- Fragmentos de código.
- Capturas de la aplicación.

Deberá incluir:

1. Resumen del proceso seguido.
2. Decisiones principales.
3. Herramientas, librerías y plantillas utilizadas.
4. Uso de inteligencia artificial y forma de validación.
5. Pruebas implementadas y razón de su elección.
6. Lista de funcionalidades terminadas.
7. Lista de funcionalidades incompletas o problemas conocidos.
8. Tiempo aproximado invertido.
9. Liga al repositorio público.

Buscamos explicaciones claras y concretas. La extensión del documento no se evaluará positivamente por sí sola.

8. Tareas adicionales opcionales

Las tareas adicionales permiten demostrar conocimientos complementarios. No realizarlas no disminuirá la calificación del alcance principal.

Por cada tarea adicional terminada podrás agregar una página al límite del PDF.

- Sin tareas adicionales: máximo cuatro páginas.
- Con una tarea adicional: máximo cinco páginas.
- Con las dos tareas adicionales: máximo seis páginas.

Las páginas adicionales deberán utilizar también Arial de 10 puntos e interlineado estándar.

Adicional 1: aplicación desplegada

Pública la aplicación en una plataforma de tu elección e incluye:

- URL pública.
- Plataforma utilizada.
- Explicación breve de cómo realizaste la implementación.
- Consideraciones o limitaciones relevantes.

La aplicación deberá poder revisarse sin solicitar credenciales adicionales a quien la evalúe.

Adicional 2: propuesta de base de datos

Explica brevemente cómo conectarías esta aplicación con una base de datos.

Puedes mencionar:

- Qué información almacenarías.
- Qué tablas o entidades utilizarías.
- Cómo se relacionarían.
- Qué tecnología utilizarías.
- Qué cambios serían necesarios en frontend y backend.

No debes implementar la base de datos. Evaluaremos especialmente la claridad, precisión y capacidad de comunicar la propuesta de manera breve.

9. Evaluación

La evaluación considerará:

- Cumplimiento de los requerimientos.
- Funcionamiento del registro y login.
- Claridad y utilidad del dashboard.
- Integración entre frontend y SnailPay.
- Manejo de respuestas del API, errores y timeout.
- Persistencia del perfil y saldo.
- Uso de React, Express y TypeScript.
- Organización y calidad del código.
- Buenas prácticas y seguridad.
- Pruebas implementadas.
- Presentación y experiencia de usuario.
- Claridad del PDF.
- Uso responsable y documentado de herramientas.
- Capacidad para explicar y modificar la solución.

No buscamos una solución perfecta. Nos interesa conocer cómo priorizas, cómo tomas decisiones y cuál es la calidad del trabajo que puedes producir dentro de un alcance y tiempo razonables.